Distributed Brute-Force Attack Detection Using Apache Spark & HDFS

Submitted To:

Muhammad Irfan Anjum

Submitted By:

Muneeb Hussain L1F22BSCS0344

M.Naveed Iqbal L1F21BSCS0230

Syed Moiz Ahmad L1S22BSCS0744

M.Omar Zahid L1F22BSCS0028

M.Ahad Farooq L1F22bscs0423

1. Project Abstract & Architecture

Modern enterprise infrastructures generate millions of security events per second, resulting in massive streams of unstructured data. Traditional relational database management systems (RDBMS) fail to scale under this velocity and volume, creating significant processing bottlenecks and delaying critical threat identification.

This project implements a distributed Big Data solution tailored for automated cybersecurity threat intelligence, specifically targeting **Real-Time Brute-Force Attack Detection**. The developed system completely decouples the storage and compute layers to maximize efficiency. By combining the fault-tolerant, distributed framework of **Apache Hadoop (HDFS)** with the high-speed, in-memory processing capabilities of **Apache Spark (PySpark)**, the pipeline automates log ingestion, filters out normal network noise, tokenizes raw string matrices, and aggregates attack frequencies to deliver a prioritized Threat Intelligence report in seconds.

2.System Architecture & Data Flow

The structural design of the platform is engineered as a clean, decoupled pipeline divided into four clear tiers: Data Generation, Distributed Ingestion, In-Memory Processing, and Reporting.

1. **Data Generation Layer:** A deterministic Python script simulates real-time network traffic, generating 10,000 time-series log entries. It uses mathematical modulo sequencing ($i \pmod 5$ and $i \pmod 7$) to systematically inject aggressive login_failed brute-force footprints amid standard network traffic.
2. **Distributed Storage Layer (HDFS):** The raw unstructured file (big_auth.log) is ingested into the Hadoop Distributed File System. The **NameNode** manages the directory filesystem metadata and maps out block allocations, while the **DataNode** physically hosts the raw log data partitions, ensuring the data is decoupled from local machine hardware constraints.
3. **Cluster Compute Layer (YARN & Spark Engine):** Controlled by the YARN **ResourceManager**, Apache Spark parallelizes the computation. Spark automatically divides the HDFS dataset into independent partitions and maps them across available CPU cores. Each core processes its chunk in parallel, executing string filtering (matching the login_failed token), column extraction (isolating the target IP_Address), and a distributed group-by shuffle operation to compile the attack sums.
4. **Data Export & Reporting Layer:** The finalized risk-priority dataset is displayed live on the console terminal and simultaneously exported back to HDFS as an optimized CSV structure. This output serves as immediate, actionable intelligence that can be fed directly into firewalls to automate defensive barriers.

2.System Prerequisites & Environment Setup

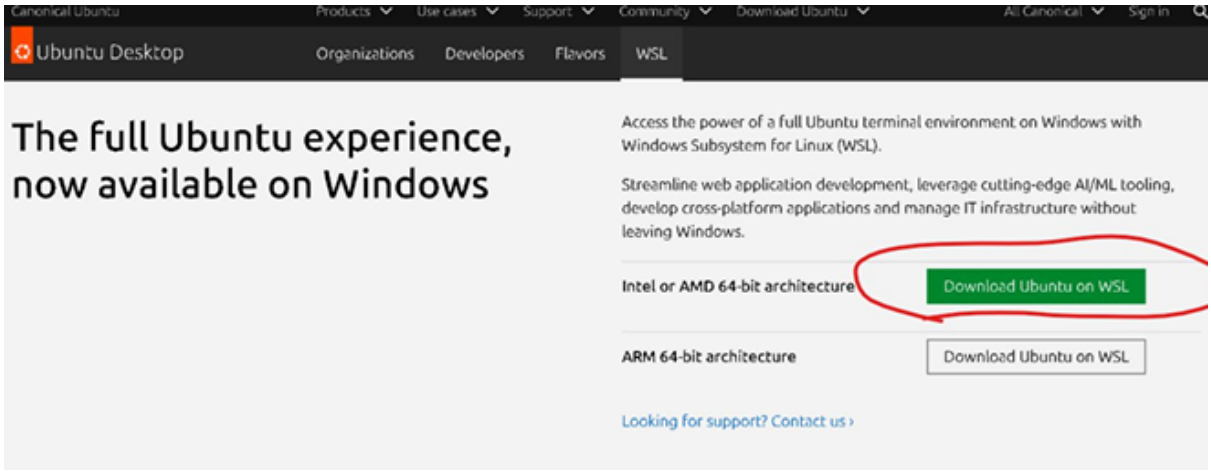
Before deploying the automated cybersecurity pipeline, the target machine must fulfill the following software and environmental requirements:

A. Core Software Requirements

- **Operating System:** Ubuntu 20.04 LTS (or higher) running natively or via **Windows Subsystem for Linux (WSL2)**.
- **Java Development Kit:** OpenJDK 8 or 11 (Apache Hadoop requires Java 8/11 to run its background daemons; higher versions may cause compatibility errors).
- **Apache Hadoop:** Version 3.x configured in Pseudo-Distributed Mode.
- **Apache Spark:** Version 3.x pre-integrated with Hadoop.
- **Python Runtime:** Python 3.8 (or higher) with the pyspark library installed.

B. Environment Setup

Download Ubuntu on WSL:



Once downloaded

- Right-click on the ZIP file
- Click Extract All
- Click Extract
- Double-click the new extracted folder
- Look for a file named similar to: ubuntu.exe
- Run Ubuntu for the First Time (VERY IMPORTANT)
- Double-click ubuntu.exe
- Ablack window will open
- Wait patiently (1–3 minutes)
  Create Linux Username
- Ubuntu will now ask:
- Enter new UNIX username e.g Abdullah

- Set a password
- You'll see something like this:
- abdullah@DESKTOP-QH91UQM

STEP 1: Move the .gz File to a Simple Location

- Open File Explorer
- Goto Downloads
- Right-click on:
- ubuntu-24.04.3-wsl-amd64.gz
- Click Cut 6. Goto:
- C:\Ubuntu File inside C:\Ubuntu

STEP 2: Click Start Menu

- Type:
- Windows Terminal
- Click Windows Terminal

àWindows Terminal open

STEP 3: Open CommandPromptTab

- In Windows Terminal
- Click the ▼ arrow
- Select Command Prompt
- Command Prompt tab active

STEP 4: Goto Ubuntu Folder

- cd C:\Ubuntu
- Press Enter
- C:\Ubuntu> shown

Install Ubuntu using WSL (MOST IMPORTANT STEP)

- Type this exact command:
- wsl--import Ubuntu-24.04 C:\Ubuntu\Installed C:\Ubuntu\ubuntu-24.04.3-wsl-amd64.gz
- Press Enter
- Wait 1–2 minutes
- Installation running

You are inside Ubuntu (WSL) now.

STEP 1: Create a Normal Ubuntu User

Hadoop should NOT be run as root.

Type this command:

- adduser abdullah
- Press Enter
- Ubuntu will ask:
- New password:
- Type a password (invisible)
- Press Enter
- Re-type password → Enter
- Then press Enter for all other questions until it finishes

STEP 2: Give User Admin Permission

- Type:
- usermod-aG sudo abdullah
- Press Enter

STEP 3: Switch to the New User

- Type:
- su- abdullah
- Press Enter
- You should now see: abdullah@DESKTOP-QH91UQM:~$

STEP 4: Update Ubuntu (MANDATORY)

- Type:
- sudo apt update
- Press Enter
- Enter password when asked
- Wait until it completes

Install Java (JDK)

- sudo apt update
- sudo apt install openjdk-11-jdk-y

STEP 1: Goto HomeDirectory

- Type:
- cd ~
- Press ENTER

STEP 2: Download Hadoop

- Type exactly:
- wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
- Press ENTER
- Wait for download to finish

STEP 3: Extract Hadoop File

- Type:
- tar-xvzf hadoop-3.3.6.tar.gz
- Press ENTER

STEP 4: Rename Hadoop Folder

- Type:
- mv hadoop-3.3.6 hadoop
- Press ENTER

STEP 5: Check Hadoop Folder Exists

- Type:
- ls
- Press ENTER
- You should see:
- hadoop

STEP 6: Set Hadoop Environment Variables

- Open bash configuration file:
- nano ~/.bashrc
- Press ENTER

STEP 7: ADDTHESELINES(DONOTCHANGESPELLING)

- Scroll to the BOTTOM of the file and paste exactly:

```
# Hadoop Environment Variables

export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64

export HADOOP_HOME=/home/abdullah/hadoop

export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin

export HADOOP_CONF_DIR=$HADOOP_HOME/etc/Hadoop
```

STEP 8: Save &Exit

What to press:

- Press CTRL + O→ ENTER
- Press CTRL + X

STEP 9: Apply Changes

- Type:
- source ~/.bashrc

- Press ENTER
- Label: APPLY environment variables

STEP 10: Verify Hadoop Installation

- What to do:
- Type:
- hadoop version
- Press ENTER
- Expected output:
- Hadoop 3.3.6

Setup SSH

Hadoop uses SSH to start services. Even on a single machine, SSH should work without asking

Do this:

- sudo apt install openssh-server-y
- ssh-keygen-t rsa-P ""-f ~/.ssh/id_rsa
- cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
- chmod 600 ~/.ssh/authorized_keys

Installing Spark Locally

- Download Spark Visit https://spark.apache.org/downloads.html and download the latest pre-built package (e.g., for Hadoop 3.3 and later).
- Extract and Configure
- tar-xzf spark-3.5.0-bin-hadoop3.tgz
- mv spark-3.5.0-bin-hadoop3 /usr/local/spark

Set Environment Variables

- Add to your .bashrc or .zshrc:
- export SPARK_HOME=/usr/local/spark
- export PATH=$PATH$:SPARK_HOME/bin

Verify Installation

- spark-shell
- A Scala shell should start, confirming successful installation.

PySpark Setup

- For Python users, install PySpark via pip:
- pip install pyspark
- To verify: from pyspark.sql import SparkSession
- spark = SparkSession.builder.appName("test").getOrCreate()
- print(spark.version)

3. Persistent Configuration

To prevent Windows Subsystem for Linux (WSL) from dropping temporary states and crashing the NameNode upon terminal closures, a persistent local metadata store must be configured.

Step 1: Create Permanent Target Paths

Execute the following command in your Linux/WSL terminal to establish a permanent storage directory:

- **mkdir -p ~/hadoop_store**

Step 2: Configure Hadoop Core Environment

Open the Hadoop core configuration file:

- **nano $HADOOP_HOME/etc/hadoop/core-site.xml**

  Ensure the <configuration> block matches the following exact structure to anchor HDFS blocks onto the permanent folder:

```
<configuration>
        <property>
                <name>hadoop.tmp.dir</name>
                <value>/home/abdullah/hadoop_store</value>
        </property>
        <property>
                <name>fs.defaultFS</name>
                <value>hdfs://localhost:9000</value>
         </property>
 </configuration>
```

  Save and close the editor via Ctrl+O then Enter then Ctrl+X.

4. The Deployment & Replication Guide

Follow these chronological phases to generate data, load the cluster, and execute the standalone Spark application.

Phase 1: Initialize System Daemons

Start the background communication tunnels and cluster engines:

- **sudo /etc/init.d/ssh start**
- **start-dfs.sh**
- **start-yarn.sh**

```
abdullah@Shadow:/mnt/c/Users/muham$ sudo /etc/init.d/ssh start
start-dfs.sh
start-yarn.sh
[sudo: authenticate] Password:
Starting ssh (via systemctl): ssh.service.
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Shadow]
Starting resourcemanager
Starting nodemanagers
abdullah@Shadow:/mnt/c/Users/muham$
```

Verify all engines are running cleanly by executing :

- **jps**

  The output must contain: NameNode, DataNode, SecondaryNameNode, ResourceManager, and NodeManager.

```
abdullah@Shadow:/mnt/c/Users/muham$ jps
4178 NodeManager
4035 ResourceManager
3785 SecondaryNameNode
3529 DataNode
4588 Jps
3390 NameNode
abdullah@Shadow:/mnt/c/Users/muham$
```

Phase 2: One-Time Master Initialization

(Note: If you have already formatted your permanent directory previously, skip this phase completely).

- **hdfs namenode -format**

Phase 3: Automated Log Generation

Create a script named generate_logs.py to systematically generate a controlled, big-data dataset containing **10,000 security logs** embedded with mathematical brute-force sequences ($i \pmod 5$ and $i \pmod 7$):

- **nano generate_logs.py**

  Paste this source code inside:

```
import random
import datetime
 ips = ["192.168.1.50", "10.0.0.15", "185.220.101.5", "172.16.0.5",
"192.168.1.22"]
 statuses = ["login_success", "login_failed", "login_failed", "login_failed"]
 users = ["user_root", "user_admin", "user_abdullah", "guest"]
with open("big_auth.log", "w") as f:
        base_time = datetime.datetime.now()
        for i in range(10000):
                ip = random.choice(ips)
                if i % 5 == 0:
                        ip = "185.220.101.5"
                elif i % 7 == 0:
                        ip = "10.0.0.15"
                status = random.choice(statuses) if ip not in ["185.220.101.5",
                "10.0.0.15"] else "login_failed"
                user = random.choice(users)
                timestamp = base_time + datetime.timedelta(seconds=i)
                f.write(f"{timestamp.strftime('%Y-%m-%d %H:%M:%S')} IP
                {ip} {status} {user}\n")

print("Successfully generated 10,000 security log entries in big_auth.log!")
```

```
  GNU nano 8.7.1                  generate_logs.py
import random
import datetime

ips = ["192.168.1.50", "10.0.0.15", "185.220.101.5", "172.16.0.5", "192.168>
statuses = ["login_success", "login_failed", "login_failed", "login_failed"]
users = ["user_root", "user_admin", "user_abdullah", "guest"]

with open("big_auth.log", "w") as f:
    base_time = datetime.datetime.now()
    for i in range(10000):
        ip = random.choice(ips)
        # Force specific IPs to behave like a massive brute-force attacker
        if i % 5 == 0:
            ip = "185.220.101.5"  # Attacker 1
        elif i % 7 == 0:
            ip = "10.0.0.15"      # Attacker 2

        status = random.choice(statuses) if ip not in ["185.220.101.5", "10>
        user = random.choice(users)
        timestamp = base_time + datetime.timedelta(seconds=i)
        f.write(f"{timestamp.strftime('%Y-%m-%d %H:%M:%S')} IP {ip} {status>
print("Successfully generated 10,000 security log entries in big_auth.log!")
```

Run the generator script:

- **python3 generate_logs.py**

```
abdullah@Shadow:/mnt/c/Users/muham$ python3 generate_logs.py
Successfully generated 10,000 security log entries in big_auth.log!
abdullah@Shadow:/mnt/c/Users/muham$
```

To view the data inside the generated file, run this:

- **hdfs dfs -cat /user/abdullah/logs/big_auth.log**

Phase 4: Cluster Data Ingestion

Migrate the generated unstructured dataset into the HDFS storage fabric:

- **hdfs dfs -mkdir -p /user/abdullah/logs**
- **hdfs dfs -put -f big_auth.log /user/abdullah/logs/**

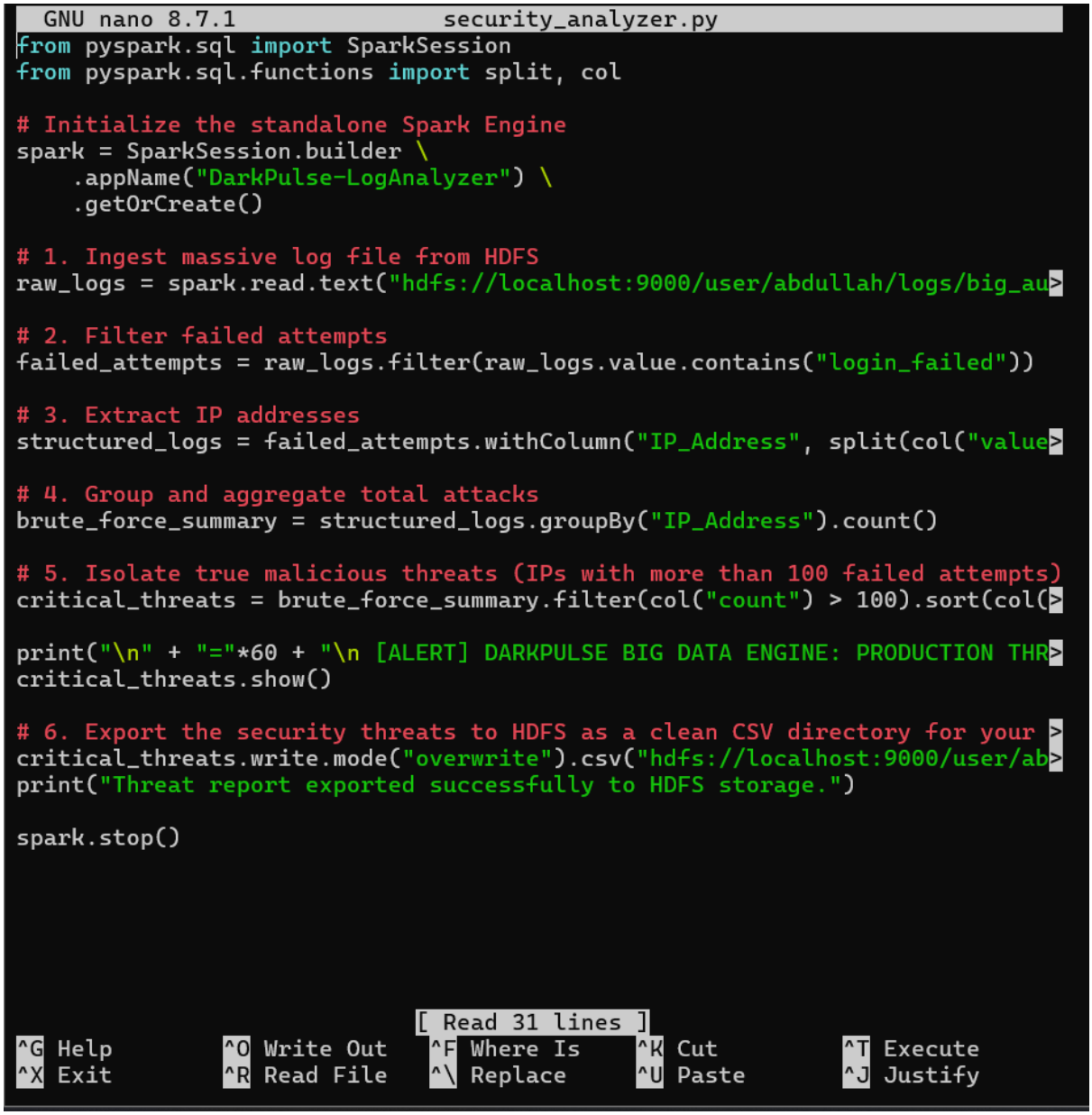
Phase 5: The Analytics Engine Setup

Create the standalone Spark application file responsible for parsing strings, extracting attack vectors, and aggregating statistical counts completely in memory:

- **nano security_analyzer.py**

  Paste this processing script inside:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import split, col
spark = SparkSession.builder \
        .appName("DarkPulse-LogAnalyzer") \
        .getOrCreate()
raw_logs=spark.read.text("hdfs://localhost:9000/user/abdullah/logs/big_auth.log
")
failed_attempts = raw_logs.filter(raw_logs.value.contains("login_failed"))
structured_logs = failed_attempts.withColumn("IP_Address", split(col("value"),
" ").getItem(3))
brute_force_summary = structured_logs.groupBy("IP_Address").count()
critical_threats = brute_force_summary.filter(col("count") >
100).sort(col("count").desc())
print("\n" + "="*60 + "\n [ALERT] DARKPULSE BIG DATA ENGINE:
PRODUCTION THREAT REPORT \n" + "="*60) critical_threats.show()
critical_threats.write.mode("overwrite").csv("hdfs://localhost:9000/user/abdulla
h/reports/threat_alerts")
print("Threat report exported successfully to HDFS storage.")
spark.stop()
```



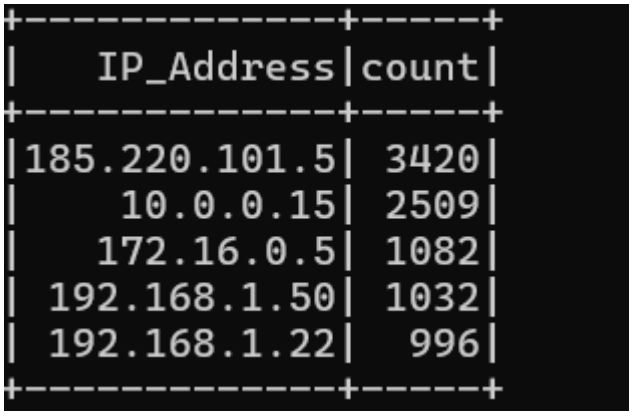
5. Execution & Execution Testing

To execute the processing pipeline, submit the standalone application script to the active YARN resource manager cluster:

- **spark-submit security_analyzer.py**

Expected Output Results

The pipeline will cleanly convert 10,000 messy log lines into a structured, prioritized Security Operations database view:



IP_Address	count
185.220.101.5	3420
10.0.0.15	2509
172.16.0.5	1082
192.168.1.50	1032
192.168.1.22	996

Always terminate cluster nodes cleanly when closing down your workstation to avoid metadata corruptions:

- **stop-yarn.sh**
- **stop-dfs.sh**

6. Conclusion & Future Scope

This project successfully demonstrates the deployment of a high-speed, distributed Big Data pipeline engineered for automated cybersecurity threat intelligence. By decoupling storage and computation using Apache Hadoop (HDFS) and Apache Spark, the system overcomes the hardware limitations and processing bottlenecks of traditional relational databases.

Through the use of deterministic log simulation and parallel in-memory computing, the standalone PySpark application effectively filters out ambient network noise, tokenizes unstructured strings, and isolates critical brute-force attack vectors in a matter of seconds. The resulting live threat report provides security operations centers (SOCs) with immediate, actionable intelligence to automate firewall defensive barriers.

Future Enhancements

To scale this architecture into an enterprise-ready system, the following modules can be integrated:

1. **Real-Time Stream Processing:** Transitioning from batch file analysis to live stream monitoring by replacing static HDFS file inputs with Apache Kafka or Spark Structured Streaming.

2. **Automated Firewall Remediation:** Scripting an automated webhook that takes the final output table of critical IP addresses and pushes them directly into a Linux IPTables or Cloud Firewall rule to block the attackers automatically.
3. **Machine Learning Classification:** Integrating Spark MLlib to analyze historical login behavior patterns, allowing the system to flag sophisticated, low-and-slow brute-force attacks that bypass traditional threshold counters.